Desenvolvimento de Sistemas Web II

O contrato gRPC, a API chamando o Go, containers e implantação

DIM0547 — Turma 01 · Sprint 2 · aula em vídeo

Prof. Fernando · UFRN · 2026.2

Onde paramos

```
26/10    Go com testes: tipos, funções como valores, interfaces, ponteiros, erros, io.Writer
passo 13  o domínio do serviço de avisos: Avisador, Canal, goroutines, context, erros com nome
```

Passo	Hoje
14	O contrato: Protocol Buffers e Buf
15	O servidor gRPC em Go
16	A API (Ktor e Quarkus) chama o serviço Go, com prazo, e continua funcionando sem ele
17	Tudo em containers: Dockerfiles, health checks e `docker compose up` do zero
18	Implantação: configuração por ambiente, migração por comando, imagens no GHCR, Render e Neon

Branch `aula`: `./passo.sh 13`.

Passo 14

O contrato: Protocol Buffers e Buf

O contrato `.proto`

```
syntax = "proto3";

package tarefas.avisos.v1;                  // a versão faz parte do nome

option go_package = "avisos/gen/tarefas/avisos/v1;avisosv1";
option java_package = "br.ufrn.exemplo.tarefas.avisos.v1";
option java_multiple_files = true;

// Envia avisos sobre tarefas.
service AvisosService {
  // Avisa, em todos os canais ao mesmo tempo, que uma tarefa foi criada. ...
  rpc Avisar(AvisarRequest) returns (AvisarResponse);
}

// O pedido de aviso.
message AvisarRequest {
  int64 tarefa_id = 1;                      // o número identifica o campo no binário
  string titulo = 2;
}
```

- `service` declara as operações do contrato (os RPCs, como `Avisar`); `message` declara os dados que trafegam (o pedido e a resposta)
- Um arquivo, três linguagens: o Go, o Kotlin e o Java geram o código a partir dele
- Números de campo nunca mudam e nunca são reaproveitados: é o que mantém a compatibilidade
- Mudança incompatível vira `v2`, ao lado da `v1`

📖 **Ref.** Protocol Buffers — proto3

Buf: conferir e gerar

```yaml
# protos/buf.yaml
version: v2
lint:
  use:
    - STANDARD
    - COMMENTS   # tudo comentado
```

```yaml
# protos/buf.gen.yaml
version: v2
plugins:
  - remote: buf.build/protocolbuffers/go:v1.36.12
    out: ../avisos-go/gen
    opt: paths=source_relative
  - remote: buf.build/grpc/go:v1.6.0
    out: ../avisos-go/gen
    opt: paths=source_relative
```

```
cd exemplos/protos
buf lint          # sem saída: o contrato segue as regras (nomes, pacote com versão, comentários)
buf generate      # escreve avisos-go/gen/tarefas/avisos/v1/*.pb.go
```

- `buf.yaml` diz como **conferir** (as regras de lint); `buf.gen.yaml` diz como **gerar** (quais plugins e para onde)
- Plugins remotos: nada de `protoc` instalado na máquina
- `buf lint` vai para o CI: a rubrica pede o contrato verde lá
- Os stubs de Kotlin e Java são gerados pelo Gradle e pelo Quarkus, no build

📖 **Ref.** Buf — lint rules

Passo 15

O servidor gRPC: status, prazo, health e reflection

O adaptador: do contrato para o domínio

```go
type Servidor struct {
    avisosv1.UnimplementedAvisosServiceServer   // métodos novos no contrato não quebram o build
    avisador *avisos.Avisador
}

func (s *Servidor) Avisar(ctx context.Context, req *avisosv1.AvisarRequest) (*avisosv1.AvisarResponse, error) {
    entregas, err := s.avisador.Avisar(ctx, avisos.Aviso{TarefaID: req.GetTarefaId(), Titulo: req.GetTitulo()})
    if errors.Is(err, avisos.ErrAvisoInvalido) {
        return nil, status.Error(codes.InvalidArgument, err.Error())   // erro do domínio → código gRPC
    }
    ...                                                                  // entregas → mensagens do contrato
}
```

O `Servidor` embute o `UnimplementedAvisosServiceServer` gerado pelo grpc-go: se o contrato ganhar uma operação nova, o build não quebra e a operação responde `UNIMPLEMENTED`. O método traduz o domínio para o contrato e usa `status.Error` para mapear cada erro do domínio num código gRPC.

Situação	Código gRPC	Equivalente HTTP
entrada inválida	`INVALID_ARGUMENT`	400
prazo vencido	`DEADLINE_EXCEEDED`	504
serviço fora do ar	`UNAVAILABLE`	503
erro interno	`INTERNAL`	500

O prazo atravessa a rede

```go
// O deadline do cliente chega no ctx do servidor. O servidor encerra os envios 50 ms
// antes, para que a resposta, com as entregas parciais, ainda chegue a tempo.
if prazo, ok := ctx.Deadline(); ok {
    var cancelar context.CancelFunc
    ctx, cancelar = context.WithDeadline(ctx, prazo.Add(-folga))
    defer cancelar()
}
```

```
cliente com prazo de 300 ms
  └─► servidor encerra em 250 ms ─► push (50 ms) e chat (150 ms) entregues; email (400 ms) não
        └─► resposta chega antes dos 300 ms: o cliente recebe a lista
```

- Sem a folga, o servidor esperaria até o fim e o cliente veria só `DEADLINE_EXCEEDED`
- O prazo viaja no cabeçalho `grpc-timeout`: é possível vê-lo no log do cliente

📖 **Ref.** gRPC — Deadlines

O servidor e a demonstração

`cmd/avisos/main.go`: registra o serviço, o *health check* padrão e o *reflection*; encerra com `GracefulStop` ao receber SIGTERM.

O *health check* gRPC responde a uma chamada padrão (`grpc.health.v1.Health`) dizendo se o serviço está de pé, para o `docker compose` e a plataforma saberem quando ele está pronto; o *reflection* deixa o `grpcurl` descobrir os serviços sem ter o `.proto`.

```
go run ./cmd/avisos                                  # escuta na 50051

grpcurl -plaintext localhost:50051 list              # o reflection lista os serviços
grpcurl -plaintext -d '{"tarefa_id": 7, "titulo": "Estudar gRPC"}' \
  localhost:50051 tarefas.avisos.v1.AvisosService/Avisar
grpcurl -plaintext -max-time 0.3 -d '{"tarefa_id": 7, "titulo": "Estudar gRPC"}' \
  localhost:50051 tarefas.avisos.v1.AvisosService/Avisar      # email volta não entregue
grpcurl -plaintext -d '{"tarefa_id": 7, "titulo": " "}' \
  localhost:50051 tarefas.avisos.v1.AvisosService/Avisar      # Code: InvalidArgument
grpcurl -plaintext localhost:50051 grpc.health.v1.Health/Check   # SERVING
```

Testes do servidor: `bufconn`, uma conexão em memória que passa pela pilha gRPC inteira, sem abrir porta.

gRPC ou REST?

	REST com JSON	gRPC com Protobuf
Contrato	OpenAPI, gerado ou escrito	`.proto`, a fonte de onde o código é gerado
Formato	texto (JSON), legível	binário, compacto
Transporte	HTTP/1.1 ou 2	HTTP/2
Erros	status HTTP e *problem details*	códigos gRPC
Bom para	APIs públicas, navegador, apps	serviço falando com serviço, dentro do sistema

> No projeto, as duas convivem: REST na API que o cliente usa; gRPC entre a API e o serviço Go.

📖 **Ref.** gRPC — Introduction

Passo 16

A API chama o Go

Uma porta nova no domínio

Ktor — `dominio/Tarefa.kt`

```kotlin
fun interface Avisador {
    suspend fun avisar(tarefa: Tarefa): Boolean
}
```

Quarkus — `dominio/Avisador.java`

```java
public interface Avisador {
    boolean avisar(Tarefa tarefa);
}
```

Na rota (ou no recurso), depois de criar:

```kotlin
val criada = repositorio.adicionar(nova)
avisador.avisar(criada)            // até 1 s; se falhar, a tarefa continua criada
```

- O domínio não sabe que existe gRPC: devolve `false` quando não conseguiu, sem exceção
- O teste de arquitetura passa a proibir `io.grpc` e `com.google.protobuf` no domínio

O adaptador no Ktor: stub com corrotinas

```
class AvisadorGrpc(private val canal: ManagedChannel, private val prazo: Duration = 1.seconds) : Avisador {
    private val stub = AvisosServiceCoroutineStub(canal)          // gerado do .proto

    override suspend fun avisar(tarefa: Tarefa): Boolean = try {
        val resposta = stub
            .withDeadlineAfter(prazo.inWholeMilliseconds, TimeUnit.MILLISECONDS)   // sempre com prazo
            .avisar(avisarRequest {                                  // DSL gerada: lambda com receptor
                tarefaId = tarefa.id.toLong()
                titulo = tarefa.titulo
            })
        log.info("aviso da tarefa {}: {}", tarefa.id, resposta.entregasList.map { "${it.canal}=${it.entregue}" })
        true
    } catch (e: StatusException) {                                   // UNAVAILABLE, DEADLINE_EXCEEDED...
        log.warn("aviso da tarefa {} não enviado: {}", tarefa.id, e.status.code)
        false
    }
}
```

`Aplicacao.kt`: `AvisadorGrpc.para(System.getenv("AVISOS_ENDERECO") ?: "localhost:50051")`, fechado no `ApplicationStopped`.

O adaptador no Quarkus: stub injetado

```java
@ApplicationScoped
public class AvisadorGrpc implements Avisador {

    @GrpcClient("avisos")                       // o Quarkus cria o stub a partir da configuração
    AvisosServiceBlockingStub stub;

    @Override
    public boolean avisar(Tarefa tarefa) {
        try {
            stub.withDeadlineAfter(1, TimeUnit.SECONDS)
                .avisar(AvisarRequest.newBuilder().setTarefaId(tarefa.id()).setTitulo(tarefa.titulo()).build());
            return true;
        } catch (StatusRuntimeException e) {
            LOG.warnf("aviso da tarefa %d não enviado: %s", tarefa.id(), e.getStatus().getCode());
            return false;
        }
    }
}
```

```
quarkus.grpc.clients.avisos.host=${AVISOS_HOST:localhost}
quarkus.grpc.clients.avisos.port=${AVISOS_PORTA:50051}
```

Stub bloqueante: o recurso já roda numa thread de trabalho.

De onde vêm os stubs

Ktor — `build.gradle.kts`

```
plugins {
  id("com.google.protobuf") version "0.10.0"
}
sourceSets {
  main { proto { srcDir("../protos") } }
}
protobuf {
  // protoc e os geradores grpc (Java)
  // e grpckt (Kotlin, com corrotinas);
  // builtin "kotlin": a DSL avisarRequest { }
}
```

Quarkus — `pom.xml`

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-grpc</artifactId>
</dependency>

<!-- em <properties>: o contrato
     fica fora do projeto -->
<quarkus.grpc.codegen.proto-directory>
  ${project.basedir}/../protos
</quarkus.grpc.codegen.proto-directory>
```

Um contrato, em `exemplos/protos/`, para os três projetos. Ktor: o BOM `grpc-bom` alinha as versões de `io.grpc`.

Degradação: a API sem o serviço Go

```
go run ./cmd/avisos                                          # terminal 1
cd ../ktor-tarefas && docker compose up -d && ./gradlew run   # terminal 2
curl -s -o /dev/null -w "%{http_code} em %{time_total}s\n" -X POST localhost:8080/tarefas \
  -H 'Content-Type: application/json' -d '{"titulo":"Estudar gRPC"}'
```

Serviço Go	Resposta da API	No log da API
no ar	`201` em ≈ 0,5 s	`aviso da tarefa 5: [email=true, push=true, chat=true]`
parado	`201` em ≈ 0,02 s	`aviso da tarefa 6 não enviado: UNAVAILABLE`
lento demais	`201` em ≈ 1 s (o prazo)	`não enviado: DEADLINE_EXCEEDED`

- A tarefa é o que importa; o aviso é acessório. Por isso a falha do aviso não vira erro HTTP
- Sem prazo, um serviço lento prenderia cada requisição pelo tempo que ele quisesse

Testes do cliente gRPC

```
// Um servidor gRPC em memória, com a resposta que o teste quiser (grpc-inprocess)
val avisador = avisadorPara("lento", object : AvisosServiceCoroutineImplBase() {
    override suspend fun avisar(request: AvisarRequest): AvisarResponse {
        delay(5_000)                    // bem mais que o prazo de 300 ms do teste
        return avisarResponse { }
    }
})
assertFalse(avisador.avisar(Tarefa(1, "Estudar gRPC")))   // e volta em menos de 2 s
```

Teste	Ktor	Quarkus
cliente: rápido, lento além do prazo, fora do ar	3	—
rota: criar avisa; aviso que falha não impede a criação	2	2
total da suíte	16	13

Passo 17

Tudo em containers

Dockerfiles em dois estágios

Go — 29 MB

```
FROM golang:1.26 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build \
    -o /avisos ./cmd/avisos

FROM gcr.io/distroless/static-debian12
COPY --from=build /avisos /avisos
USER nonroot
ENTRYPOINT ["/avisos"]
```

Ktor — cerca de 600 MB

```
FROM eclipse-temurin:25-jdk AS build
WORKDIR /src
COPY protos protos
COPY ktor-tarefas ktor-tarefas
WORKDIR /src/ktor-tarefas
RUN ./gradlew --no-daemon -q installDist -x test

FROM eclipse-temurin:25-jre
COPY --from=build \
  /src/ktor-tarefas/build/install/ktor-tarefas /app
CMD ["/app/bin/ktor-tarefas"]
```

- O primeiro estágio compila; o segundo só leva o resultado. Ferramentas de build não vão para produção
- `CGO_ENABLED=0` gera um binário estático, sem depender de bibliotecas do sistema, e a `distroless` é uma imagem mínima, sem shell: por isso os 29 MB
- O contexto do build da API é `exemplos/`, porque ela precisa de `protos/`

Health checks e o `docker compose`

Serviço	Health check
PostgreSQL	`pg_isready`
API Ktor	`GET /health`
API Quarkus	`GET /q/health` (extensão `quarkus-smallrye-health`, que também verifica o banco)
Go	`avisos saude`: o próprio binário chama o health check gRPC (a imagem não tem shell nem `curl`)

```
  api-ktor:
    build: { context: ., dockerfile: ktor-tarefas/Dockerfile }
    environment:
      DB_URL: jdbc:postgresql://banco:5432/tarefas
      AVISOS_ENDERECO: avisos:50051          # dentro do compose, o nome é o endereço
    depends_on:
      banco: { condition: service_healthy }
      avisos: { condition: service_healthy }
```

`depends_on` com `condition: service_healthy` faz a API esperar o banco e o `avisos` ficarem saudáveis antes de subir; dentro da rede do compose, o nome do serviço é o endereço (`avisos:50051`).

```
cd exemplos && docker compose up --build      # e: docker compose --profile quarkus up
docker compose ps                             # banco, avisos e api-ktor: healthy
```

Passo 18

Implantação

A mesma imagem, configurada pelo ambiente

Variável	Desenvolvimento	No compose	Em produção
`DB_URL`	`localhost:5432`	`banco:5432`	a connection string do Neon
`AVISOS_ENDERECO`	`localhost:50051`	`avisos:50051`	o endereço interno do serviço Go
`PORT`	8080	8080	a porta que a plataforma define

- Nada de configuração de ambiente no código: a imagem é a mesma em todo lugar
- Senhas e connection strings são **segredos**: entram no painel da plataforma, nunca no repositório

Migração por comando, antes de a versão nova entrar no ar:

```
/app/bin/ktor-tarefas migrar      # aplica as migrações e termina
```

Do commit à imagem publicada

`exemplos/implantacao/ci-e-imagens.yml` (modelo; no projeto, fica em `.github/workflows/`):

```yaml
jobs:
  testes:                                  # em todo push e PR
    steps:
      - uses: bufbuild/buf-action@v1       # buf lint no contrato
      - run: go test ./...
      - run: ./gradlew test                # inclui Testcontainers: o runner tem Docker
  imagens:
    needs: testes
    if: github.ref == 'refs/heads/main'    # só na main, e só com os testes verdes
    permissions: { packages: write }
    steps:
      - uses: docker/build-push-action@v6
        with:
          tags: ghcr.io/${{ github.repository }}/api-ktor:${{ github.sha }}
```

O job `testes` roda em todo push e PR; o job `imagens` roda só na `main` e depois dos testes verdes, para não publicar uma versão quebrada.

Cada imagem leva a etiqueta do commit: voltar a uma versão é apontar para a etiqueta anterior.

📖 **Ref.** GitHub — Publishing Docker images

No ar: Render e Neon, e um limite importante

`exemplos/implantacao/render.yaml`: a API como web service, o serviço Go como serviço privado, o banco no Neon.

Plano gratuito do Render (consultado em 15/10)	Consequência
serviço privado não tem instância gratuita	o Go num serviço privado custa
web service gratuito não recebe tráfego pela rede privada	o Go como web service gratuito não é alcançável pela API
o endereço público não encaminha gRPC	não é possível chamar o Go pela internet
`preDeployCommand` só em plano pago	no gratuito, a API migra na subida

> Na entrega, o critério **Sistema no ar** pede a API e o banco no ar. O serviço Go roda junto da API no `docker compose`; pô-lo no ar é opcional, registrado numa ADR.

📖 **Ref.** Render — Private network · Render — Free instances

O que a Sprint 2 cobra

Critério	Peso	O que mostrar
Microsserviço Go	25%	responsabilidade justificada (ADR); Go idiomático, `context` e erros
Contrato e gRPC	25%	`.proto` comentado, `buf lint` verde no CI, stubs gerados, chamada com prazo e erro tratado, teste de integração
Sistema no ar	25%	API e banco gerenciado no ar, URL no README, migração por task ou CI, segredos fora do repositório, health check, API que funciona sem o Go (o Go no ar é opcional)
Ambiente e testes	15%	`docker compose up` do zero; unitários, integração e teste de arquitetura, verdes local e no CI
Documentação	10%	README completo (roda em menos de 10 min), OpenAPI por URL, 2 ADRs novas, `LICENSE`

Prova em **11/11**; *daily meetings* em 23 e 25/11; **entrega em 30/11**.

> **Prova (11/11):** presencial, no laboratório, no horário da aula, no **Multiprova**, com questões fechadas sobre as Sprints 0 e 1. É permitida **uma folha A4 manuscrita, frente e verso**; nenhum outro material pode ser consultado.

Onde estudar depois

- Leitura da sprint: `leituras/web2-s2.md`, capítulos 6 a 9
- Passos: `exemplos/avisos-go/PASSOS.md` · branch `aula`, tags `passo-16` a `passo-18`
- gRPC — Kotlin quick start · Quarkus — gRPC clients
- Docker — multi-stage builds · Compose — services
- Render — Blueprint YAML · Neon — Connect from any application